

Latency-Sensitive Service Chaining with Security Isolation Constraints

Yannick Carlinet[†], Nancy Perrot[†], Laurent Valeyre[†], Krzysztof Bocianiak*,
Wojciech Niewolski*, Aleksandra Podlasek* and Jean-Philippe Wary[†]

**Orange Poland, Innovation Division
Warsaw, Poland*

*†Orange, Innovation Division
Paris area, France*

Email: *firstname.lastname@orange.com*

Abstract—The envisioned requirements for 6G include, among others, lower environment impact, imperceptible end-to-end latency, high resiliency, security and privacy. They can only be achieved by combining both compute and communications capabilities of the whole network: core, edge, access. This is the reason why Multi-access Edge Computing (MEC) is one of the key enablers in 6G, by offering computing capabilities near the end users. This will enable the deployment of latency-sensitive services. However, deployment of services in the edge poses some security challenges. This work aims at solving orchestration issues, such as how to deal with service deployment while meeting service-specific requirements, while at the same time meeting security isolation requirements, in a multi-cluster environment. We could build an orchestration engine, with an advanced algorithm to compute optimal placement of micro-services, integrated in a state-of-art emulation platform with industry best practice. This demonstrates the feasibility in practice of our approach.

Index Terms—Edge Computing, Latency-sensitive Service, Service Chaining, Security Isolation

I. INTRODUCTION

Multi-access edge computing (MEC) is a distributed computing architecture that brings computing and storage resources closer to the network edge. It enables applications and services to be hosted and processed at the network edge, rather than in centralized data centers or cloud platforms. By bringing computing closer to the end-users, MEC can reduce network latency, improve application performance, and support emerging use cases that require low latency and high bandwidth, such as augmented reality, virtual reality, or autonomous vehicles. MEC can also enable new business models and revenue streams for network operators and service providers, by providing value-added services, such as content caching, video optimization, or network slicing.

The advantages of edge computing include:

Reduced latency: Edge computing reduces the time it takes for data to be processed, analyzed and acted upon, as the data is processed locally. **Improved reliability:** Edge computing reduces the dependence on a centralized system, making the overall system more resilient to network failures or disruptions. **Enhanced security:** With edge computing,

sensitive data can be processed locally, reducing the risk of data breaches and improving data privacy. **Lower bandwidth requirements:** Edge computing reduces the amount of data that needs to be transmitted over the network, as only important data is sent to the cloud or data center. This reduces the bandwidth requirements and the associated costs. **Cost savings:** Edge computing reduces the need for expensive network infrastructure and reduces the cost of transmitting large amounts of data to a centralized location.

However, there are also some challenges that need to be addressed, including:

Resource Constraints: Edge nodes typically have limited computing and storage resources compared to centralized data centers, which can limit the types and scale of applications that can be deployed on edge nodes.

Security: MEC introduces new security challenges, such as securing distributed computing resources and protecting sensitive data at the edge of the network. More specifically, an exploit can enable a rogue (or compromised) application to have access to non-authorized parts of RAM, thus allowing to steal or poison the data of the other applications running on the same physical host. In a cloud environment, applications with greater security requirements would run on dedicated hardware. However, this is no longer possible in MEC because of the scarcity of resources.

Orchestration: MEC requires the management and orchestration of resources across multiple edge nodes and clusters, while meeting the requirements of service chains, including security and end-to-end latency constraints.

In this paper, we focus on the orchestration challenges in MEC. In particular, the paper brings the following contributions:

- Description and modelling of an optimization problem, called SLSCP (Security and Latency Service Chaining Problem) ;
- Algorithms that provide solutions for SLSCP ;
- Set-up of a platform that emulates a realistic Edge Computing environment ;
- Integration of the algorithms with the Kubernetes orchestrators of each cluster, demonstrating the feasibility of

multi-clusters orchestration, with service-specific requirements.

Section II describes related work, while in Section III, an overview of the MEC architecture and the emulation platform are given. Section IV details the security isolation model. Section V gives a formal description of SLSCP, ways to solve it, and the results of numerical experiments.

II. RELATED WORK

The problem of placing chains of services in the network with latency constraints have been extensively studied [11], [14], [10], [5], [4], [7]. But not all have considered specifically the architecture of MEC ([12], [1]). Some works deal specifically with other constraints or objectives, like energy [3], or security [6]. However, none has tackled yet service-specific requirements like latency and security jointly, and implemented the resulting algorithms on a production-ready environment for validation.

III. THE MEC EMULATION PLATFORM

The MEC architecture we consider in this work is based upon the edge infrastructure described in [2], where small Data Centers (DC) are distributed down to the Cell Sites, as summarized in Fig. 1.

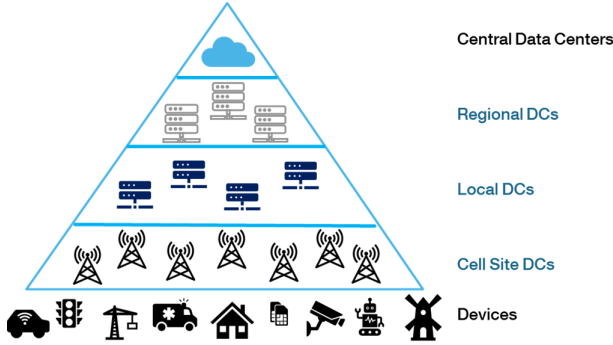


Fig. 1. Proposed Edge Infrastructure Model

To demonstrate the feasibility in practice of our approach, we have setup an emulation platform that mimic the MEC architecture of Fig. 1. Each DC is materialized by a Kubernetes cluster in our platform.

For practical reasons, the whole platform infrastructure is virtualized. It is built upon best practice tools in industry. Firstly, it is based on LXC¹. Then, the infrastructure building phase is automated thanks to Terraform² and Ansible³. Fig. 2 illustrates the internal structure of a cluster. HAProxy and keepalived keep the high access availability for flows, whether administration, customer, or inter-cluster flows. All clusters have their own private network and they share a common global DNS. Its role is to forward all external DNS requests to the right local DNS. Then, each local DNS synchronizes all local applications with the external-DNS applications.

¹<https://linuxcontainers.org/lxc>

²<https://www.terraform.io>

³<https://www.ansible.com>

We have installed Istio⁴ to manage the global service mesh (multi-clusters) by selecting the architecture 'multi-primary on different network'. This architecture proposes for each cluster its own control plane, so that the clusters are independent from each other. We have also installed Kiali⁵ to benefit from a feature-rich monitoring framework (cf. Fig. 3 for an illustration). Finally, we use ArgoCD⁶ for automated deployment of service chains in the platform (cf. Fig. 3).

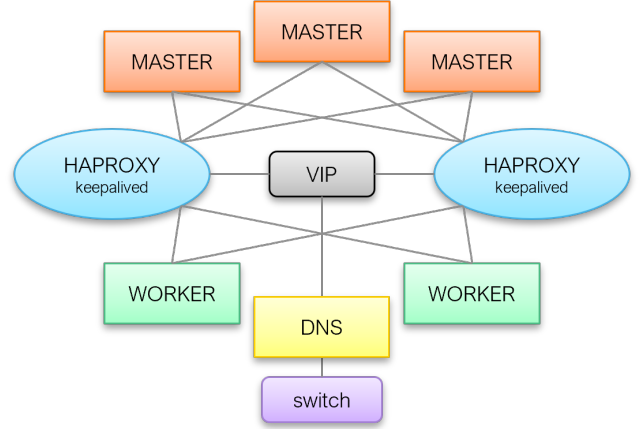


Fig. 2. Structure of one cluster

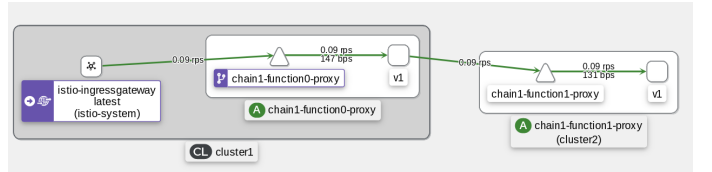


Fig. 3. Kiali over Istio: monitoring of a Service Chain over 2 clusters

The platform is also used to perform measurements on the communication latency between the micro-services of a Service Chain, thanks to Jaeger⁷, a micro-service tracing tool.

Overall, the platform is composed of 3 clusters, each having 3 masters and 7 worker nodes (only 2 are shown on the figures for clarity purpose).

The integration of the placement algorithms with the Kubernetes orchestrators (each cluster has its own orchestrator) is done through the Kubernetes affinity mechanism. In Kubernetes, worker nodes can be given labels, chosen arbitrarily by the administrator. When micro-services are deployed, it is possible to specify affinity rules with the node labels. The principle of our approach is to give each node a unique label. When the placement of micro-services are determined (cf. the algorithm description in Section V), we give affinity rules to the orchestrators of the different clusters, so that they comply with the placement we have computed. In order to keep the

⁴<https://istio.io>

⁵<https://kiali.io>

⁶<https://argoproj.github.io/cd>

⁷<https://www.jaegertracing.io>

benefit of automatic scaling provided by Kubernetes, for each micro-service, we specify affinity with several worker nodes, when possible.

IV. SECURITY ISOLATION MODEL

Security requirements are a part of the Service Level Agreement (SLA) between the provider of IaaS (Infrastructure as a Service) and a Service Provider. The edge Data Centers must be secured by providing isolation of resources and data, because they belong to different tenants or network slices. Indeed, there are a number of attacks that can be performed by a rogue (or compromised) slice to steal or poison data used by another slice on the same physical host (cf. [12]). However, it is not possible to provide physical isolation to all slices, because of cost-efficiency and technical issues. Instead, we propose to physically isolate sensitive slices from less secure ones.

To this end, each Service Chain defines an Isolation level as part of its requirements. This can be defined in the contracted SLA. If data processed by this service are very sensitive, the service will require a higher level of isolation. The isolation level is freely chosen by the Service Chain owner, but of course the price charged by the IaaS provider will be impacted by this choice.

In addition, each Service Chain is given a Trust level. This can be determined with a security assessment process, such as an audit, on the following criteria, for instance:

- Container image vulnerability scanning or audit
- Service development process (programming language, coding practice, libraries used, ...)
- Service Chain testing process

The Trust level is determined by the infrastructure provider. If the Service owner does not want to submit its service to the security audit, then it is given the lowest possible trust level.

The Service Chain security isolation can then be applied, by making sure that two Service Chains can be physically co-located **if, and only if, the Trust level of one chain is higher or equal than the Isolation level of the other chain**. In other words, it is ensured that sensitive chains are only co-located with secure chains.

This approach has several advantages over physically isolated slices: first and foremost, it enables cost-efficient physical isolation. Realistically, it is simply not feasible to have a physically isolated infrastructure for each sensitive services. Another advantage is that this kind of security isolation might not be needed all the time, it can be deployed when needed, thus saving resources and costs at the other times.

V. OPTIMIZATION OF MICRO-SERVICES PLACEMENT

When a Service Chain must be deployed, the administrator of the platform provides the placement algorithm with the description of the micro-services that compose the chain. The algorithm also probes the platform in order to retrieve all needed information about the worker nodes and already running services. Then the algorithm is executed, and it gives

as output the placement of micro-services that meet all the service and security isolation requirements.

A. Problem description

In the so-called Security and Latency-aware Service Chain-
ing Problem (SLSCP), we consider a set of nodes V , a set of micro-services types F , and a set of Service Chains S . A given chain $k \in S$ is composed of an ordered set of functions (or micro-services), denoted F_k . A given function $f \in F$ is characterized by a Trust T_f and Isolation level I_f (cf. Section IV), and by requirements in terms of RAM, CPU, and storage, denoted respectively R_f^{ram} , R_f^{cpu} , R_f^{sto} . The aim is to attribute a placement for each function onto a node.

Each node $u \in V$ is characterized by a cost for deploying a function on this node c_u , and its capacities in terms of RAM, CPU and storage, denoted respectively C_u^{ram} , C_u^{cpu} , C_u^{sto} .

Each Service Chain $k \in S$ has a requirement for the maximum allowed end-to-end communication latency, noted L_k . We assume the communication latency of using a link between two nodes u and $v \in V$ is constant and is noted L_{uv} . We make use of the emulation platform to perform experimental measurements of the latency of all links. The latency of the whole chain is the sum of all the links used by consecutive functions in the chain.

TABLE I
NOTATIONS SUMMARY

Sets	
V	Set of nodes (index u)
F	Set of functions (index f)
S	Set of chains (index k)
F_k	Set of functions in chain $k \in S$
Constants	
I_f	Isolation level of function f
T_f	Trust level of function f
R_f^{cpu}	Requirement in vCPU of function f
C_u^{cpu}	Capacity in vCPU of node u
R_f^{ram}	Requirement in RAM of function f
C_u^{ram}	Capacity in RAM of node u
R_f^{stor}	Requirement in storage of function f
C_u^{stor}	Capacity in storage of node u
c_u	Cost of using node u
l_{uv}	Latency induced by using link (u, v)
L_k	Maximum end-to-end latency admitted by chain k
S_{fg}	Indicator whether f and g are consecutive
B_{fg}	Indicator whether g is a backup for f
Variables	
x_{uf}	Binary indicator of function f placed on node u
y_u	Binary indicator of node u in use

Some functions are marked as backup for another one, so they cannot be both on the same node (anti-affinity constraints).

B. Formulation

The decision variables are:

$x_{uf} \in \{0, 1\}$, that takes the value 1 if function f is placed on node u , 0 otherwise, $\forall u \in V, \forall f \in F$.

$y_u \in \{0, 1\}$, that takes the value 1 if node u is used, 0 otherwise, $\forall u \in V$.

Table I sums up the notations and a compact formulation is given (cf. Equations 1-8).

$$\min \sum_{k \in S} \sum_{u, v \in V} \sum_{f, g \in S_k} x_{uf} x_{vg} S_{fg} l_{uv} \quad (1)$$

subject to

$$\sum_{f \in F} R_f^{cpu} x_{uf} \leq C_u^{cpu} y_u \quad \forall u \in V \quad (2)$$

$$\sum_{f \in F} R_f^{ram} x_{uf} \leq C_u^{ram} y_u \quad \forall u \in V \quad (3)$$

$$\sum_{f \in F} R_f^{stor} x_{uf} \leq C_u^{stor} y_u \quad \forall u \in V \quad (4)$$

$$I_f(x_{uf} + x_{uf'} - 1) \leq T_{f'} x_{uf'} \quad \forall u \in V, \forall f, f' \in F \quad (5)$$

$$\sum_{u \in V} x_{uf} = 1 \quad \forall f \in F \quad (6)$$

$$\sum_{u, v \in V} \sum_{f, g \in S_k} x_{uf} x_{vg} S_{fg} l_{uv} \leq L_k \quad \forall k \in S \quad (7)$$

$$x_{uf} + x_{ug} \leq 2 - B_{fg} \quad \forall u \in V, \forall f, g \in F \quad (8)$$

In this formulation, the objective (Eq. 1) is to minimize the overall communication latency of all chains. Eq. 2-4 are the capacity constraints. Eq. 5 enforce the security isolation constraints. Eq. 6 ensure that each micro-service is placed once, while Eq. 7 enforce the maximum latency for each service chain. Finally, Eq. 8 make sure that backup functions are not located on the same node as the function they protect.

In order to make the formulation linear, we define a new set $F^C \subset F \times F$ as follows: $\forall f, g \in F, (f, g) \in F^C \Leftrightarrow S_{fg} = 1$. Put simply, F^C is the set of couples of consecutive functions. We define a new variable as follows: $z_{uvfg} \in \{0, 1\}$, that takes the value 1 if $x_{uf} = x_{vg} = 1$, and the value 0 otherwise, $\forall u, v \in V, \forall (f, g) \in F^C$. This variable is defined only for functions that are consecutive in a chain. Equation 7 becomes:

$$\sum_{u, v \in V} \sum_{(f, g) \in F^C} z_{uvfg} l_{uv} \leq L_k \quad \forall k \in S \quad (9)$$

$$x_{uf} + x_{vg} - 1 \leq z_{uvfg} \quad \forall u, v \in V, \forall (f, g) \in F^C \quad (10)$$

$$z_{uvfg} \leq x_{uf} \quad \forall u, v \in V, \forall (f, g) \in F^C \quad (11)$$

$$z_{uvfg} \leq x_{vg} \quad \forall u, v \in V, \forall (f, g) \in F^C \quad (12)$$

C. Multi-criteria Formulations

With respect to energy savings, it is valuable to have the service chains span over the least workers as possible. In this way, when the load is low, it becomes possible to turn off unneeded workers. Therefore, in order to have the functions packed as much as possible, we introduce a cost c_u for using node u (i.e. at least one function is placed on it). This cost can be interpreted as a power consumption cost. We then define the SLSCP_cost problem as:

$$\min \sum_{u \in V} c_u y_u \quad (13)$$

$$\text{with constraints (2) - (8)} \quad (14)$$

We can then define a new problem, denoted SLSCP_2steps_cost, which consists in a two step process: first we find a solution to SLSCP_cost, which yields a minimum cost denoted c^* , then we solve SLSCP with the additional following constraint:

$$\sum_{u \in V} c_u y_u \leq c^* \quad (15)$$

It is also possible to perform the two-steps process by first minimizing the overall latency, then adding a constraint on the overall latency and minimizing the cost. This approach is denoted SLSCP_2steps_lat in the following.

D. Greedy heuristics

For comparison purpose, we also have designed a greedy heuristics, in which we perform the following steps:

- Sort the service chains, by increasing maximum allowed latency, then by decreasing number of functions
- Sort the hosts by cluster id, then by increasing trust level, then by decreasing available CPU resource.

These sorts are designed so that the chains with the highest requirements are placed first, to prevent placement failure as much as possible. Then we take each chain one by one in order, and place its functions on the first host possible, checking each host in order. Hosts are re-sorted after placing a function. If a function cannot be placed, or if a chain violates its maximum allowed latency constraint, it means greedy has failed to find a feasible solution for this instance.

We denote *greedy_rand*, a variant in which the service chains are not sorted, but are randomly shuffled instead.

E. Numerical experiments

We have implemented the algorithms described in Section V in Python [13] with Pyomo [8] and the Cplex [9] solver. We have also generated randomly a large number of service chains, with varying number of chains per instance, and number of functions per chain.

Fig. 4, 5 show the total latency vs. the total number of functions, respectively for 2, 3 and 4 service chains.

The comparison of greedy and greedy_rand indicates that sorting the service chains by decreasing maximum latency requirements does not provide any benefit, on average.

As shown on Fig. 6, the gap between greedy and the optimum is between 25% and 45%, except with low count of functions. This show that the exact optimization model allows to gain at least 25% (up to 45 %) of the average communication latency obtained with a greedy heuristics.

VI. CONCLUSION

In conclusion, in the work described in this paper, we have reach the following achievements:

- Definition, modelling and solving of the SLSCP problem
- Integration with Kubernetes orchestrator, to provide multi-cluster and service-specific orchestration
- Description of an approach for Security-by-Orchestration

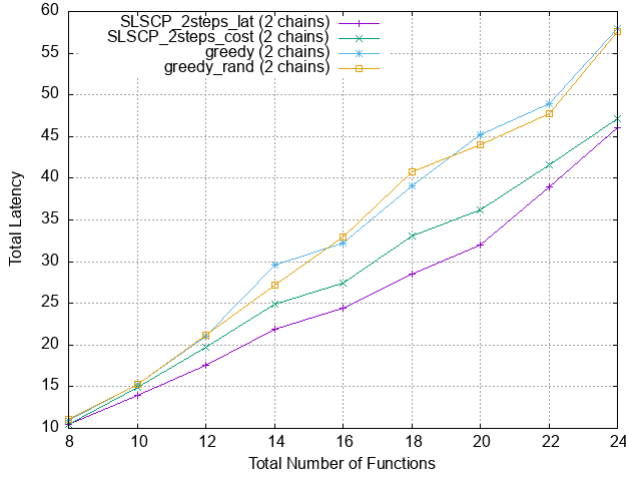


Fig. 4. Latency vs. nb of functions (2 chains)

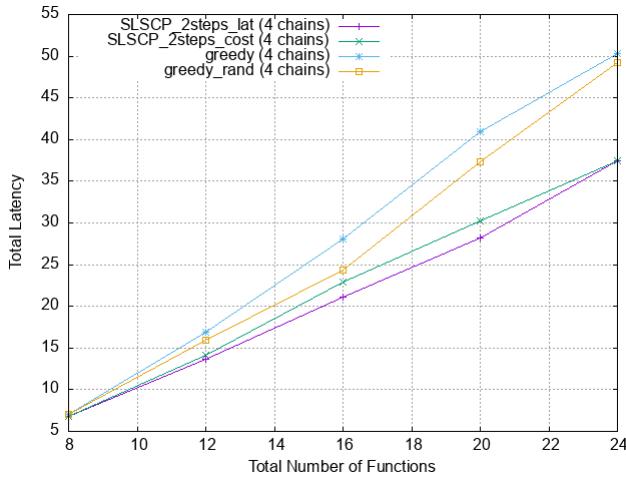


Fig. 5. Latency vs. nb of functions (4 chains)

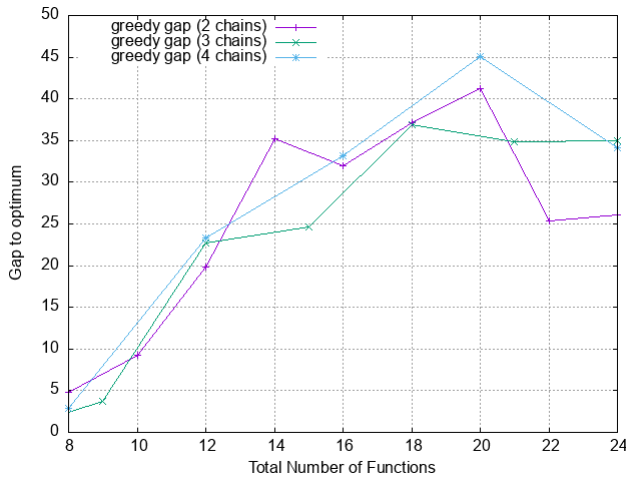


Fig. 6. Relative gap to optimum

- Setup of an emulation platform, in accordance with industry best-practice.

In consequence, we could demonstrate the feasibility of a security by orchestration approach.

Currently, the platform only supports so-called bare-metal clusters. We are now in the process of integrating AWS (Amazon Web Service) clusters into it. Afterwards, we will also integrate a GCP (Google Cloud Platform) cluster, in order to make the platform a true hybrid, multi-clusters edge computing emulation platform.

We are also investigating ways to improve the performance of the placement algorithm, with respects to execution time.

REFERENCES

- [1] T. W. Nowak al. Verticals in 5g mec-use cases and security challenges. in *IEEE Access*, 9:87251–87298, 2021.
- [2] R. Artych, K. Bocianiak, Y. Carlinet, W. Niewolski, N. Perrot, A. Podlasek, and J. Wary. Security constraints for placement of latency sensitive 5g mec applications. In *2022 9th International Conference on Future Internet of Things and Cloud (FiCloud)*, pages 40–45, Los Alamitos, CA, USA, aug 2022. IEEE Computer Society.
- [3] H. Badri, T. Bahreini, D. Grosu, and K. Yang. Energy-aware application placement in mobile edge computing: A stochastic optimization approach. in *IEEE Transactions on Parallel and Distributed Systems*, 31(4):1, April 2020.
- [4] S. Bi, L. Huang, and Y. A. Zhang. Joint optimization of service caching placement and computation offloading in mobile edge computing systems. in *IEEE Transactions on Wireless Communications*, 19(7):4947–4963, July 2020.
- [5] B. Brik, P. A. Frangoudis, and A. Ksentini. Service-oriented mec applications placement in a federated edge cloud architecture. In *Icc Ieee, editor, International Conference on Communications (ICC) pp*, pages 2020–2020. 1-6, 2020.
- [6] L. Caviglione and M. Gaggero. Multiobjective placement for secure and dependable smart industrial environments. in *IEEE Transactions on Industrial Informatics*, 17(2):1298–1306, February 2021.
- [7] L. Ferrucci et al. Latency preserving self-optimizing placement at the edge. In *Proceedings of the 1st Workshop on Flexible Resource and Application Management on the Edge*, 2020.
- [8] William E. Hart, Carl D. Laird, Jean-Paul Watson, David L. Woodruff, Gabriel A. Hackebeil, Bethany L. Nicholson, and John D. Sirola. *Pyomo—optimization modeling in python*, volume 67. Springer Science & Business Media, second edition, 2017.
- [9] ILOG, Inc. ILOG CPLEX: High-performance software for mathematical programming and optimization, 2006. <http://www.ilog.com/products/cplex/>.
- [10] R. Mahmud, A. N. Toosi, K. Ramamohanarao, and R. Buyya. Context-aware placement of industry 4.0 applications in fog computing environments. in *IEEE Transactions on Industrial Informatics*, 16(11):7004–7013, November 2020.
- [11] Ahlam Mouaci, Éric Gourdin, Ivana Ljubić, and Nancy Perrot. Two extended formulations for the virtual network function placement and routing problem. *Networks*, 03 2023.
- [12] 5G PPP White Paper. Edge computing for 5g networks. <https://bscw.5g-ppp.eu/pub/bscw.cgi/d397473/EdgeComputingFor5GNetworks.pdf>, 2021.
- [13] Guido Rossum. Python reference manual, 1995, 1995. <http://www.python.org>.
- [14] M. Wang, B. Cheng, W. Feng, and J. Chen. An efficient service function chain placement algorithm in a mec-nfv environment. *IEEE Global Communications Conference (GLOBECOM)*, 2019:1–6, 2019.